# Aula Completa: O Sistema Distribuído de Pedidos com RabbitMQ

---

## 1. O que estamos fazendo? (A Visão Geral)

Para entendermos este projeto (o TP01), imagine que a nossa turma foi contratada para construir a espinha dorsal de um grande **E-commerce**. Quando um cliente clica em "Comprar", uma cascata de eventos precisa ocorrer nos bastidores de forma confiável e independente.

Em um sistema monolítico (antigo), uma única máquina validaria o pagamento, consultaria o estoque e dispararia o e-mail. Mas, o que acontece na **Black Friday**? A máquina não aguenta e cai. O pagamento é perdido, e o cliente fica com a tela travada.

O que nós estamos fazendo é **Sistemas Distribuídos na Prática**: Separamos cada função em "um pequeno programa independente" (os Consumers) e usamos o **RabbitMQ** como o nosso correio super rápido no meio deles. Se o sistema de pagamentos cair, o RabbitMQ guarda as mensagens em uma fila até que ele volte, garantindo que nenhum dado se perca no processo.

---

## 2. A Arquitetura: Como tá funcionando?

Toda a infraestrutura do projeto simula a comunicação assíncrona entre múltiplas máquinas usando:

- **RabbitMQ Cluster (3 Nós):** Nós subimos 3 servidores RabbitMQ usando Docker (rabbit1, rabbit2 e rabbit3) para operarem em conjunto.
- **Quorum Queues (Raft Consenso):** O sistema garante *Alta Disponibilidade*. Uma mensagem entrando no Nó 1 é automaticamente copiada pros nós 2 e 3 antes dela ser confirmada como "recebida". Se o Nó 2 explodir do nada, o Nós 1 e 3 dão conta do recado e ninguém percebe.
- **Producer (A Loja):** Ele gera os eventos de "Novo Pedido" e envia para o RabbitMQ (no Topic Exchange).
- **Consumers (Os Trabalhadores):** Programas em Python que ficam "escutando" as filas. Temos trabalhadores específicos para Estoque (`consumer_stock.py`), Pagamento (`consumer_payment.py`), e Notificações (`consumer_notification.py`).

---

## 3. Entendendo os Arquivos (O que cada um faz?)

Abaixo está o detalhamento técnico do diretório do nosso trabalho:

### A Infraestrutura

- **`docker-compose.yml`**: É a receita que cria o "mundo" do nosso sistema distribuído. Ele baixa as instâncias do RabbitMQ e do Banco de Dados e as liga numa mesma rede.
- **`init_cluster.sh`**: O script responsável por pegar os 3 servidores e uni-los ("ei, vocês

3 formam o cluster agora"). Ele configura as Quorum Queues de acordo.
- **`init_db.sql`**: Um arquivo SQL executado automaticamente para criar as tabelas no PostgreSQL (Auditoria, Estoque, etc).

### A Lógica do Negócio (Python API)

- **`producer.py`**: A aplicação de frente para a loja. Ele cria formatos JSON de compras aleatórias (Ex: Cliente CUST-0102 comprou um monitor de $800) e publica isso no Exchange `orders.exchange`.
- **`base_consumer.py`**: A superclasse orientada a objetos (Herança). Para não repetir código, essa classe encapsula toda a lógica que qualquer consumidor precisa ter (ACK manual, DLQ handling e auto-reconectar ao PostgreSQL).
- **`consumer_*.py`**: São os executores práticos. O `consumer_payment.py`, por exemplo, só escuta a fila `orders.payment`. Ele atualiza a tabela de pagamentos lá no banco de imagem assíncrona. O `consumer_audit.py` possui uma Routing Key curinga (`#`) o que significa que ele consegue interceptar *todas* as mensagens do sistema.

### As Ferramentas Finais

- **`benchmark.py`**: Como testar se o sistema distribuído é melhor que o monolítico? Esse script injeta, digamos, 100.000 mensagens na arquitetura, lança múltiplos Consumers e cronometra como o tempo de vazão (Throughput) cai e a eficiência aumenta. Ele plota um gráfico.
- **`dashboard.py`** & **`templates/index.html`**: Para não lermos logs feios no terminal, o Dashboard se liga diretamente na API do Node Principal do RabbitMQ, extrai os números por segundo via SSE (Server-Sent Events) e os desenha numa tela interativa de um site (HTML/Chart.js).

---

## 4. O Nosso Resultado Final

Como projeto, o que extraímos no final?

1. Uma prova viva do funcionamento de **resiliência computacional**: podemos parar um nó no meio da execução que tudo continuou salvando no banco.
2. Demonstração de **Consistência Eventual**: não garantimos que a notificação foi no mesmo momento do clique da compra, mas garantimos que ela ocorreu asíncronamente instantes depois.
3. Demonstração de **Throughput medido**: Através da geração visual com os gráficos (matplotlib e web dashboard), obtemos evidências empíricas de quantos clusters e processos precisamos para esvaziar filas do tamanho de uma Black Friday.